Network Message / Header Formats

HTTP · DNS · TCP · UDP · IP · DHCP · Ethernet · ARP — explained in simple English

> **Big idea:** Every protocol sends data in a fixed "envelope" layout. The **header** is the address/instruction part of the envelope. The **data (payload / body)** is the actual letter inside. Knowing the header fields tells you *who sent it, who gets it, and how to handle it*.
> **Layers order (top → bottom):** Application (HTTP, DNS, DHCP) → Transport (TCP, UDP) → Network (IP, ARP) → Data Link (Ethernet).

Application layer

1. HTTP Message Format

HTTP is how your browser talks to a website. There are **two** message types: **Request** (browser asks) and **Response** (server answers). Both are plain text.

Legend: sp = Space, cr = Carriage Return, lf = Line Feed (cr + lf together = "end of this line", like pressing Enter).

REQUEST message (Browser → Server)

Request line

Method	sp	URL	sp	Version	cr lf

Header lines (many)

Header name	:	sp	Value	cr lf
• • • (more header lines)				

Blank line

cr lf (marks "headers finished")

Body

Variable number of lines (present only in some messages)

RESPONSE message (Server → Browser)

Status line

Version	sp	Status code	sp	Phrase	cr lf

Header lines (many)

Header name	:	sp	Value	cr lf

• • • (more header lines)

Blank line

cr lf

Body

Variable number of lines
(present only in some messages)

Fields explained

Part / Field	Simple meaning	Example
Method	What action the browser wants. GET = "give me this page", POST = "I'm sending you data (like a form)", PUT = "replace/upload this", DELETE = "remove this", HEAD = "just the headers, no body".	`GET`
URL	Which resource (page/file) you want on the server.	`/index.html`
Version	Which HTTP version is being used.	`HTTP/1.1`
Status code	A number telling the result. 200 = OK, 301 = moved, 404 = not found, 500 = server error.	`200`
Phrase	Human-readable text for the status code.	`OK`, `Not Found`
Header name : Value	Extra information as pairs. E.g. `Host: www.site.com`, `User-Agent: Chrome`, `Content-Type: text/html`, `Content-Length: 1200`.	`Host: example.com`
sp / cr / lf	Just separators. **sp** = space between items; **cr+lf** = end of a line.	—
Blank line	An empty line (only cr lf). It tells the receiver: "Headers are over, body starts next."	—
Body	The actual data: HTML page in a response, or form data in a POST request. Many GET requests have **no body**.	HTML text

Remember: Request starts with a *Request line* (Method URL Version). Response starts with a *Status line* (Version Code Phrase). Everything else is the same layout.

Application layer · uses UDP (port 53)

2. DNS Message Format

DNS is the "phone book of the internet": it turns a name like `google.com` into an IP address. The **query** and the **response** use the **same format**.

0 16 31 (bits)

Identification 16 bits	Flags 16 bits
Number of questions	Number of answer RRs
Number of authority RRs	Number of additional RRs

↑ These first 3 rows = **12-byte header** | ↓ below = the variable-length sections

Questions (variable number of questions) – name & type fields for a query
Answers (variable number of resource records) – RRs in response to query
Authority (variable number of resource records) – records for authoritative servers
Additional information (variable number of resource records) – extra "helpful" info that may be used

Header fields (the 12 bytes)

Field	Simple meaning
Identification (16 bits)	A random ID number. The reply copies the same number so the client knows *which question* this answer belongs to.
Flags (16 bits)	Small on/off switches: **QR** (0 = query, 1 = response), **Opcode** (type of query), **AA** (answer is from an authoritative server), **TC** (message was truncated), **RD** (recursion desired – "please find it for me"), **RA** (recursion available), **Rcode** (error code, 0 = no error).
Number of questions	How many entries are in the Questions section (usually 1).
Number of answer RRs	How many records are in the Answers section.
Number of authority RRs	How many records are in the Authority section.

Number of additional RRs	How many records are in the Additional section.

RR = Resource Record (one piece of DNS data, like "google.com → 142.250.x.x").

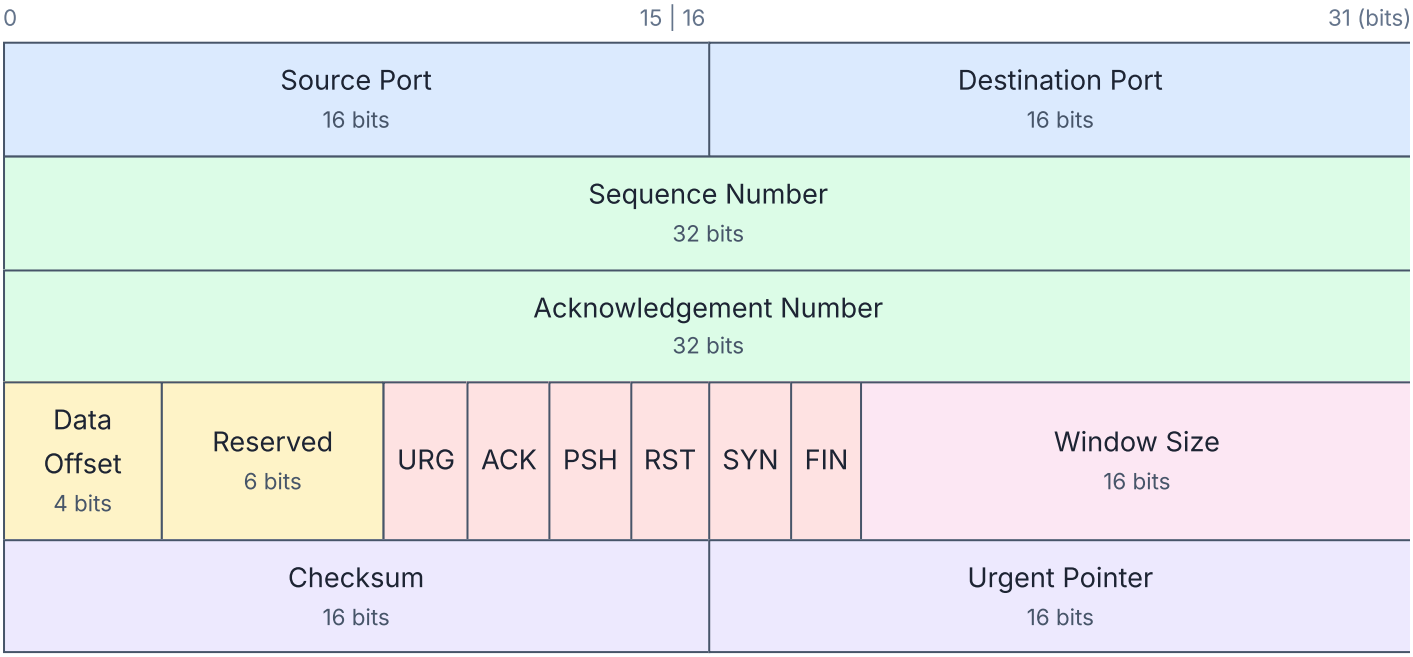
Body sections

Section	What it holds
Questions	The name being asked (e.g. `www.example.com`) and the type (A = IPv4 address, AAAA = IPv6, MX = mail server, NS = name server, CNAME = alias).
Answers	The actual answer record(s) to your question, e.g. the IP address.
Authority	Tells which name servers are the official owners ("authoritative") of that domain.
Additional information	Bonus info that might save you another query, e.g. the IP of the name server mentioned in Authority.

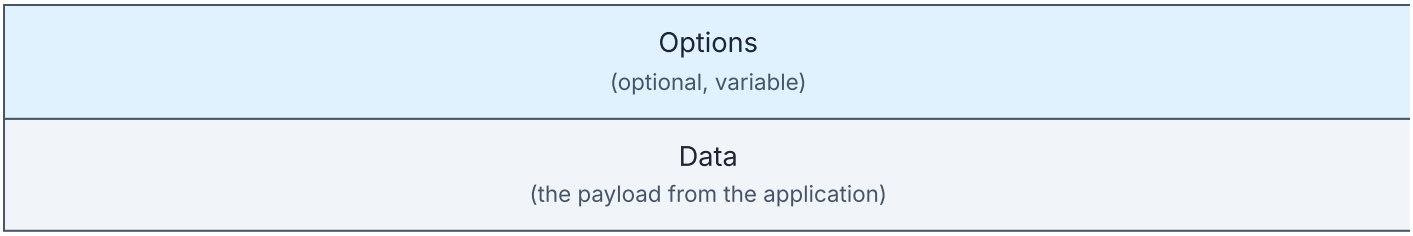
Transport layer

3. TCP Packet (Segment) Structure

TCP is **connection-oriented** and **reliable**: it sets up a connection first, numbers every byte, checks for errors, and resends lost data. Used by HTTP, HTTPS, FTP, email.



↑ Above = fixed **20-byte header**



Fields explained

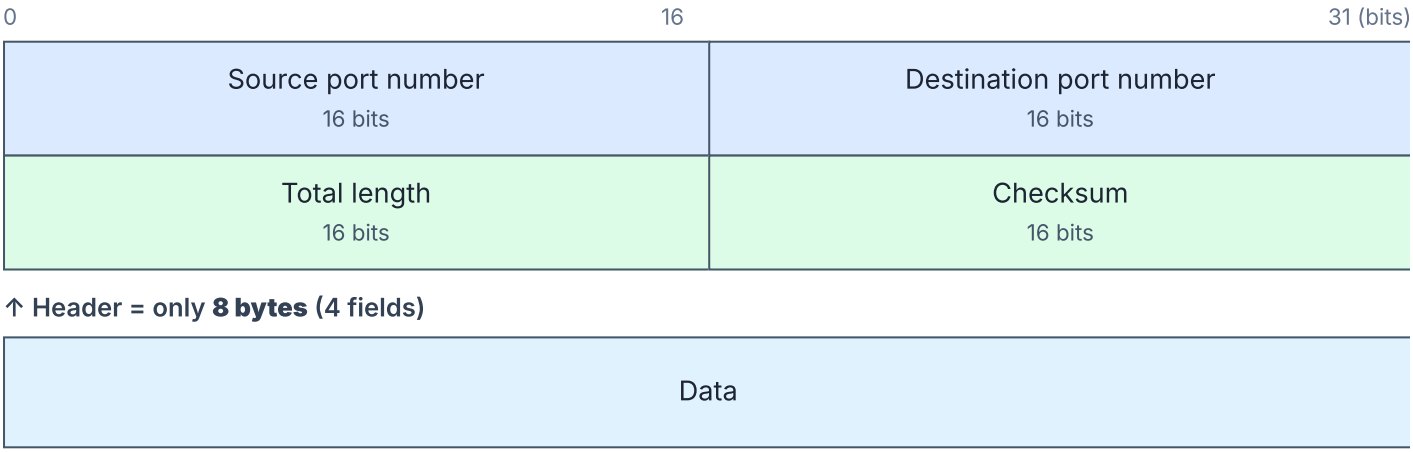
Field	Simple meaning
Source Port (16)	The "door number" on the **sender's** computer that this data comes from (identifies the sending app).
Destination Port (16)	The "door number" on the **receiver's** computer it should go to. E.g. 80 = HTTP, 443 = HTTPS, 25 = SMTP.
Sequence Number (32)	Number of the first data byte in this segment. Lets the receiver put pieces back in the right order.
Acknowledgement Number (32)	"I got everything up to here; send me this byte number next." Proves delivery.
Data Offset (4)	Length of the TCP header (in 4-byte words). Tells where the data begins.

Reserved (6)	Not used now; kept empty (zeros) for the future.
URG	Urgent flag – this segment has urgent data (see Urgent Pointer).
ACK	Acknowledgement flag – the Ack Number field is valid.
PSH	Push flag – deliver this data to the application immediately, don't wait.
RST	Reset flag – break the connection abruptly (something went wrong).
SYN	Synchronize flag – start a new connection (used in the 3-way handshake).
FIN	Finish flag – I'm done sending; close the connection politely.
Window Size (16)	Flow control: how many bytes the receiver can accept right now ("my buffer space").
Checksum (16)	Error check over header + data. If it doesn't match, the segment is damaged and dropped.
Urgent Pointer (16)	Points to where the urgent data ends (only used when URG = 1).
Options	Optional extras, e.g. maximum segment size, window scaling, timestamps.
Data	The actual payload (e.g. part of a web page).

Transport layer

4. UDP Packet Structure

UDP is **connectionless** and **simple**: just send it, no handshake, no guarantee. It's fast. Used by DNS, DHCP, video calls, online games, streaming.



Fields explained

Field	Simple meaning
Source port (16)	Port of the sending application.
Destination port (16)	Port of the receiving application (e.g. 53 for DNS, 67/68 for DHCP).
Total length (16)	Size of the whole UDP packet = 8-byte header + data.
Checksum (16)	Basic error check to see if the packet got damaged.

> **Answer to the question in your notes:** A UDP header has **4 fields** – source port, destination port, length and checksum.

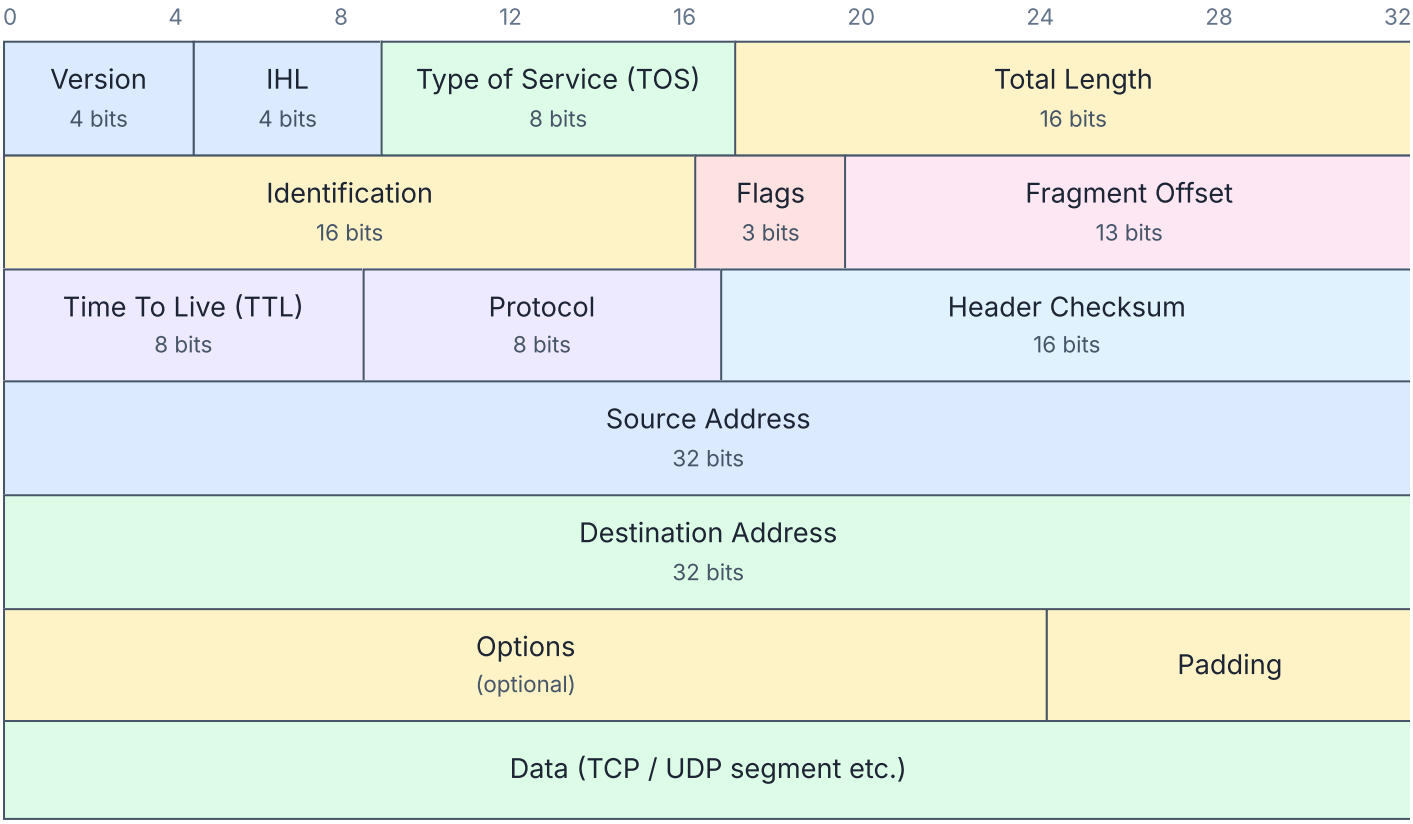
TCP vs UDP – quick compare

	TCP	UDP
Connection	Yes (handshake)	No
Reliable?	Yes – resends lost data, keeps order	No guarantee
Header size	20 bytes (min)	8 bytes
Speed	Slower	Faster
Used for	Web, email, file transfer	DNS, DHCP, video, games

Network layer

5. IPv4 Header

IP gives every device an address and moves packets from the source computer to the destination computer across many networks (routers). It is "best effort" – no delivery guarantee.



Fields explained

Field	Simple meaning
Version (4)	IP version. IPv4 → value 4.
IHL – Internet Header Length (4)	Size of the IP header in 4-byte words. Minimum is 5 (= 20 bytes). Bigger if Options are used.
Type of Service / TOS (8)	Tells routers how important/fast this packet is (priority). Split into: **Precedence** (priority level), **Delay** (want low delay), **Throughput** (want high speed), **Reliability** (want reliable path), **Reserved** (unused).
Total Length (16)	Whole packet size (header + data) in bytes. Maximum 65,535.
Identification (16)	An ID number for the packet. If a packet is cut into pieces, all pieces share the same ID so they can be joined back.
Flags (3)	**Reserved** (always 0) · **DF – Don't Fragment** (1 = don't cut this packet into pieces) · **MF – More Fragments** (1 = more pieces follow; 0 = this is the last piece).

Fragment Offset (13)	Position of this piece within the original packet, so pieces can be put in order.
Time To Live – TTL (8)	A countdown. Each router subtracts 1. At 0 the packet is thrown away. Stops packets looping forever.
Protocol (8)	What is inside the data: 6 = TCP, 17 = UDP, 1 = ICMP.
Header Checksum (16)	Error check for the *header only* (recalculated at every router because TTL changes).
Source Address (32)	IP address of the sender, e.g. `192.168.1.5`.
Destination Address (32)	IP address of the receiver.
Options + Padding	Rarely used extras (security, routing, timestamp). Padding adds zeros so the header ends on a 32-bit boundary.
Data	The payload, e.g. a TCP or UDP segment.

Application layer · uses UDP (ports 67 server, 68 client)

6. DHCP – Getting an IP Address (4 steps)

When a new device joins a network it has no IP address. DHCP automatically gives it one. Remember the word **DORA**: **D**iscover, **O**ffer, **R**equest, **A**CK.

Example: DHCP server = `223.1.2.5`; arriving client has no address yet (`0.0.0.0`).

1 **DHCP Discover** (Client → everyone, broadcast)
"Is there any DHCP server out there? I need an IP address!"
`src: 0.0.0.0, 68` · `dest: 255.255.255.255, 67` · `yiaddr: 0.0.0.0` · `transaction ID: 654`

2 **DHCP Offer** (Server → client)
"Yes, I'm here. You can use this address: 223.1.2.4 for 3600 seconds."
`src: 223.1.2.5, 67` · `dest: 255.255.255.255, 68` · `yiaddr: 223.1.2.4` · `transaction ID: 654` · `lifetime: 3600 secs`

3 **DHCP Request** (Client → server, broadcast)
"Great, I accept that address 223.1.2.4, please give it to me."
`src: 0.0.0.0, 68` · `dest: 255.255.255.255, 67` · `yiaddr: 223.1.2.4` · `transaction ID: 655` · `lifetime: 3600 secs`

4 **DHCP ACK** (Server → client)
"Confirmed! The address is yours for 3600 seconds."
`src: 223.1.2.5, 67` · `dest: 255.255.255.255, 68` · `yiaddr: 223.1.2.4` · `transaction ID: 655` · `lifetime: 3600 secs`

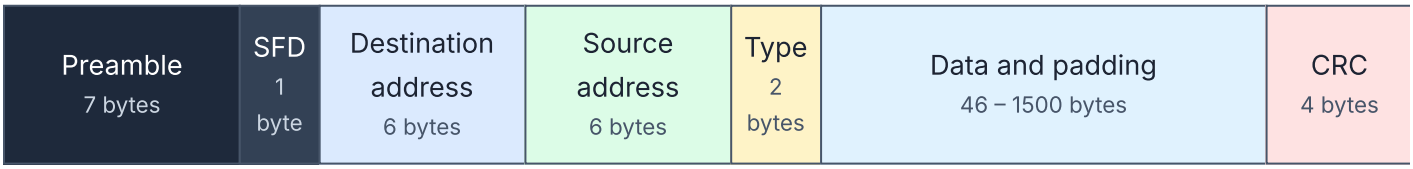
What each item in the boxes means

Item	Simple meaning
src	Sender's IP address and port number.
dest	Receiver's IP address and port. `255.255.255.255` = broadcast = "send to everyone on this network".
Port 67 / 68	67 = DHCP server port, 68 = DHCP client port.
yiaddr ("your Internet address")	The IP address being given to the client. `0.0.0.0` means "not assigned yet".
transaction ID	A number to match replies with the right request (654 for Discover/Offer, 655 for Request/ACK in the slide).
lifetime	How long the client may keep the address (lease time), here 3600 seconds = 1 hour. After that it must renew.

Data link layer

7. Ethernet Frame Format

On a local network (LAN), data travels in **frames**. The frame uses **MAC addresses** (hardware addresses burned into network cards) to reach the next device.



Preamble + SFD = "physical-layer header" (8 bytes, not counted in the frame length)

Minimum frame length = 512 bits (64 bytes) · Maximum frame length = 12,144 bits (1518 bytes)

Fields explained

Field	Simple meaning
Preamble (7 bytes)	56 bits of alternating 1s and 0s (10101010...). Like a "wake-up beep" that lets the receiver sync its clock.
SFD (1 byte)	Start Frame Delimiter = `10101011`. Says "the real frame starts right after this."
Destination address (6)	MAC address of the device that should receive this frame (e.g. `AA:BB:CC:11:22:33`).
Source address (6)	MAC address of the device that sent it.
Type (2)	What protocol is inside: `0x0800` = IPv4, `0x0806` = ARP, `0x86DD` = IPv6.
Data and padding (46–1500)	The payload (e.g. an IP packet). If data is under 46 bytes, padding is added to reach the minimum. Max 1500 bytes (this is the MTU).
CRC (4)	Cyclic Redundancy Check – error detection. If it doesn't match, the frame is damaged and discarded.

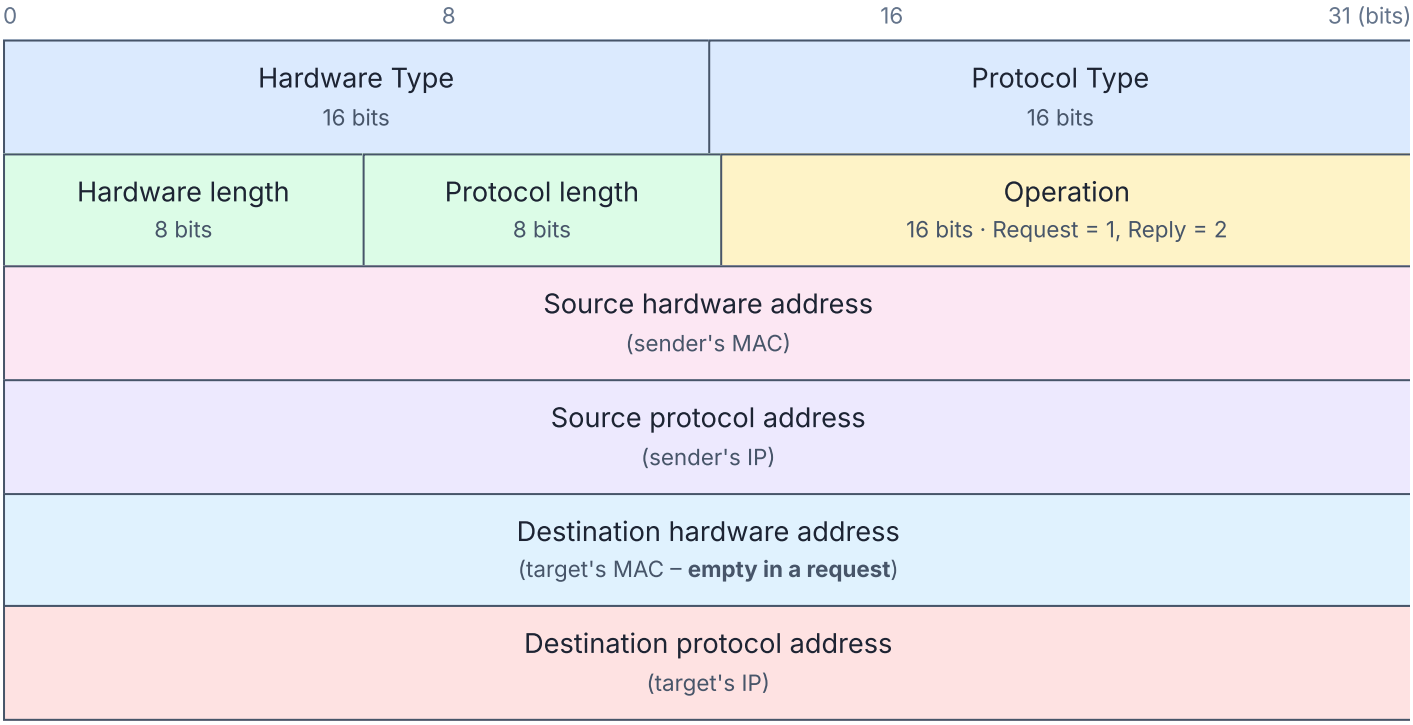
Size maths: 6 + 6 + 2 + 46 + 4 = **64 bytes** minimum frame; 6 + 6 + 2 + 1500 + 4 = **1518 bytes** maximum frame.

Between Network and Data link layer

8. ARP Protocol & Packet Format

> **What ARP does:** You know the IP address of a device, but the LAN needs its **MAC address**. ARP finds it: **IP → MAC address**. ARP takes an IP address from the IP protocol, finds the matching link-layer (MAC) address, and hands it to the data-link layer.

ARP Request: *Broadcast* – "Who has IP 192.168.1.10? Tell me your MAC!" (sent to everybody).
ARP Reply: *Unicast* – only the owner answers directly: "That's me, my MAC is ...".



Fields explained

Field	Simple meaning
Hardware Type (16)	Which type of network hardware. Ethernet = `1`.
Protocol Type (16)	Which network protocol is being resolved. IPv4 = `0x0800`.
Hardware length (8)	Size of a hardware (MAC) address in bytes = `6` for Ethernet.
Protocol length (8)	Size of a protocol (IP) address in bytes = `4` for IPv4.
Operation (16)	`1` = ARP Request, `2` = ARP Reply.
Source hardware address	MAC address of the sender of this ARP message.
Source protocol address	IP address of the sender.

Destination hardware address	MAC address of the target. In a **request it is empty (all zeros)** because that's exactly what we're trying to find. Filled in the reply.
Destination protocol address	IP address of the target we are looking for.

Tiny example

1. **Request (broadcast):** "Who has `192.168.1.10`? Tell `192.168.1.5` (my MAC = AA:AA…)". Target MAC field is empty.
2. **Reply (unicast):** "`192.168.1.10` is at MAC `BB:BB…`" – sent only to the asker.
3. The asker saves this in its **ARP cache** and now builds an Ethernet frame with the right destination MAC.

Network layer · companion of IP

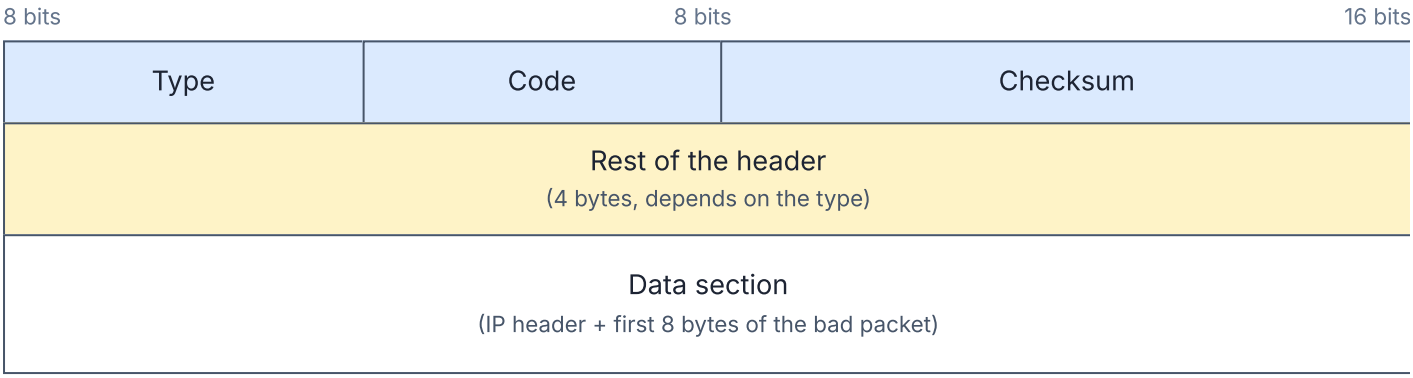
9. ICMP Protocol & Message Format

> **What ICMP does:** IP just sends packets and does not tell you if something went wrong. ICMP (Internet Control Message Protocol, version 4) is IP's helper. It fixes two weaknesses of IP: **(1) no error reporting** (if a packet is lost, nobody is told) and **(2) no way to ask questions** (like "is that host alive?"). ICMP messages travel inside IP packets.

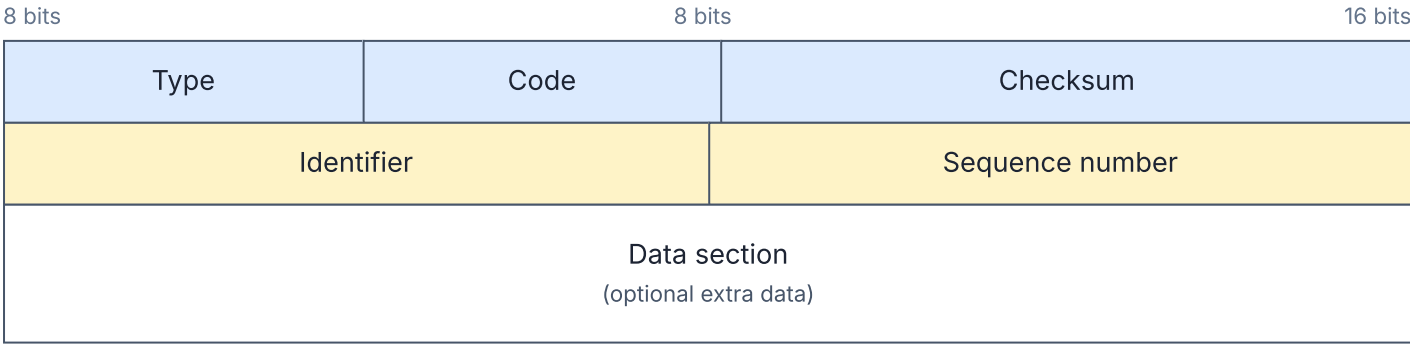
ICMP messages are of **2 types**:

1. **Error-reporting message** – a router/host says "something went wrong with your packet".
2. **Query message** – a host asks a question and gets an answer (e.g. ping).

Error-reporting message



Query message



Fields explained

Field	Simple meaning
Type (8)	The main kind of message. `8` = Echo Request (ping), `0` = Echo Reply, `3` = Destination Unreachable, `11` = Time Exceeded, `5` = Redirect, `12` = Parameter Problem, `4` = Source Quench (slow down).
Code (8)	A more detailed reason inside that type. Example: Type 3 with Code 0 = network unreachable, Code 1 = host unreachable, Code 3 = port unreachable.
Checksum (16)	Error check for the whole ICMP message. Damaged message = dropped.

Rest of the header (error messages)	4 bytes whose meaning depends on the type. Often unused (zeros), or holds extra info (e.g. the gateway address in a Redirect).
Identifier (query messages)	Number that identifies which program/process sent the request, so the reply goes back to the right one.
Sequence number (query messages)	Counts each request (1, 2, 3 ...). Lets you match each reply to its request and spot lost ones.
Data section	**Error message:** a copy of the IP header + first 8 bytes of the packet that caused the problem, so the sender can find out which packet failed. **Query message:** optional data (ping fills it with filler bytes, which the reply sends back).

Tracing ICMP packets (Ping)

ICMP packets can be traced using the `ping` command:

1. `ping 8.8.8.8` sends an **ICMP Echo Request** (Type 8, Code 0) with an identifier and sequence number.
2. The other host replies with an **ICMP Echo Reply** (Type 0, Code 0), same identifier and sequence number.
3. Ping shows the **time taken** (round-trip time) and the **TTL**. No reply means the host is down or blocked.

> **Also remember:** `traceroute` uses ICMP **Time Exceeded** (Type 11). It sends packets with TTL = 1, 2, 3 ... Each router that drops a packet (TTL hit 0) reports back, so you see every router on the path.

Quick Summary Table

Protocol	Layer	Main job	Header size
HTTP	Application	Request/get web pages (text format)	Variable (text lines)
DNS	Application	Name → IP address	12 bytes + sections
DHCP	Application	Automatically give IP address	— (4 steps: DORA)
TCP	Transport	Reliable, ordered delivery	20 bytes (min)
UDP	Transport	Fast, no-guarantee delivery	8 bytes
IP	Network	Address & route packets between networks	20 bytes (min)
ICMP	Network	Error reports & queries (ping, traceroute)	8 bytes (4 fixed + 4 rest)
ARP	Network/Link	IP → MAC address	28 bytes (IPv4/Ethernet)
Ethernet	Data link	Deliver frames inside a LAN using MAC	14 bytes + 4 CRC

Made for easy exam revision · Good luck! 🎓